

We split the training data into training set and train-dev set and test on train-dev

If the train-dev produced high error --> model overfit

If the train-dev produced low error --> data mismatch

Stop when you have a model that performs well on both train-dev and validation

Chapter 3

Classification

Evaluating Classification Models:

1-Accuracy:

If you have unbalanced dataset and you're testing if that label belongs to a certain class, if that label is only 10% of the data and every guess turns out to be the other class so this gives a 90% accuracy, but that doesn't mean that that model is performing well

So, this performance measure is not preferred performance, especially when dealing with skewed data

So, we move to

2-Confusion Matrix

Shows the number of true positive, true negatives (these are the main diagonal and is best when it is a non-zero)

Also shows false positive, false negatives (incorrectly classified)

The second part is the prediction, and the first part is is it correctly classified or not

Ex: True positive (the model predicted it as positive, and it is true positive)

3-Precision

Measures the accuracy of the positive predictions

$$TP / TP + FP$$

Not always accurate ... why?

If you have a model that shows positive if it's very confident about it (confidence score), else it shows negative class

If i only have 1 instance of that's in positive class and it correctly classified, then the precision would be 100% as

$$(1 / 1 + 0) * 100$$

That's why we use it along with another metric:

3-Recall (sensitivity)

Shows the true positive rate

$$TP / TP + FN$$

To get a better metric is to combine recall and precision:

3-F1 Score

Combines both metrics, but not using mean (as it will treat both values equally)

So it uses harmonic mean, so it gives much more weight to lower values

$$2 * (\text{precision} * \text{recall}) / \text{precision} + \text{recall}$$

So, a high f1 score would be a result if both recall and precision are high

But sometimes we don't need that kind of balance

Example: if we're building a model to detect if a video is safe for kids or not , we'll be wanting to decrease the false positive as much as we can , because we would rather reject good videos that popping up a bad video for a kid

So we'll need high precision -> low false positive

While if we're in the medical field, we'll need more false negative, we would rather check up on everyone even if they're not sick rather than missing a sick patient

So high recall -> low false negative

To control which should have a higher priority than the other : Beta Value

$$F\beta = (1+\beta^2) \cdot (\beta^2 \cdot \text{Precision}) + \text{Recall} \cdot \text{Precision} \cdot \text{Recall}$$

Beta = 1 --> equal weights

Beta < 1 --> precision has more weight

Beta > 1 --> recall has more weight

Or we can put some kind of threshold to the resulting score of a decision function

How do you decide which threshold to use? `Cross_val_predict`, `tunedthresholdclassifiercv` (uses cross-validation to get optimal threshold metric)

Another metric to measure

4-ROC Curve

Plots the true positive rate, recall (sensitivity) against false positive rate equivalent to 1-true negative rate aka

Plots recall and 1-specificity

So it measures the roc auc curve if it is = 1 then this is a good classifier, else 0.5 randomly classified

Random Forrest doesn't have decision function, but it has `predict_proba()` returns estimated class probabilities for each instance `edict_proba()` method that returns estimated class probabilities for each instance, and we can just

use the probability of the positive class as a score, so `precision_recall_curve()` will work.

Multi-class Classification:

Classifying more than 2 classes

In sk-learn classifiers like logistic regression, random forest classifier, Gaussian NB can handle multiple class classification

While others like SGDClassifier and SVC are strictly binary

To create a system that can classify multi-class is through

1-One vs all:

Trains a model for each class, comparing it with the rest. Example, if you have 5 classes classifying numbers from 1-5, you'll train 5 models

Model 1: +ve class --> 1 -ve class--> 2-5

Model 2: +ve class --> 2 -ve class --> 1,3-5

Model 3: +ve class --> 3 -ve class --> 1,2,4,5

And so on

To predict, you'll run it on all 5 models and pick the one with the highest decision score (how confident the classifier believes that the output belongs to a certain class)

Very simple and easy to implement

But, every model sees a very imbalanced dataset

2-One vs One

Train a model for each pair of classes, so you'll end up with

$$N(N-1) / 2$$

Where N is the number of classes you have

To predict, you'll run all models you have and pick the digit with the highest vote

And you'll calc its decision score too (to avoid ties) for each class

And the final decision is based on the score

Each classifier sees only 2 classes and has small dataset

Can be more accurate

But, more time to predict

3- All

SVM scale poorly with the size of dataset which means that its computational costs increase with the size of data making it impractical for large datasets

Why does this happen with SVM?

Because SVM depends on a quadratic optimization function and the cost grows with increasing the number of training samples, because its complexity is almost n^2 or n^3

So for SVM, one vs one training would be more suitable as it requires a small amount of training data for each model

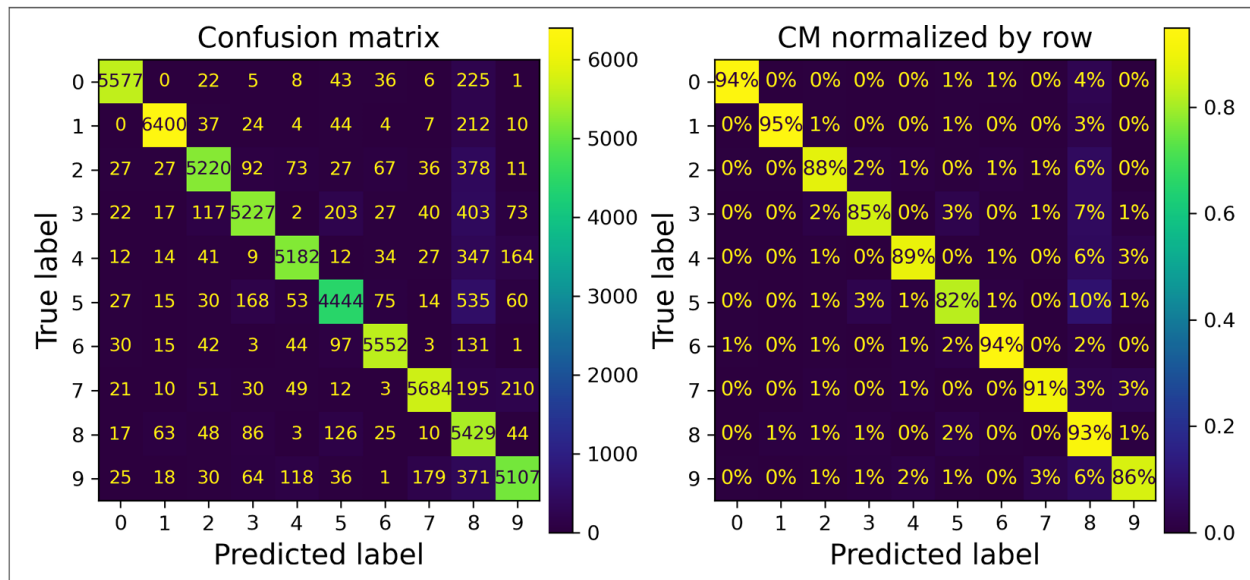
$$\frac{1}{2} * ||w||^2$$

Since SVM wants the best separating line, the one which is the furthest that's why we use the above rule

Error Analysis

After choosing a model, you want to improve it by analyzing the errors

1-Confusion Matrix



By looking at it, we'll see that the diagonals have brighter colors which means high values, and we've mentioned earlier that the diagonals show correctly classified data

But looking at number 5 in rows and columns, its slightly darker than the others which means its smaller than it

So is it smaller because it was not correctly classified or because number 5 was fewer in the dataset

How to know?

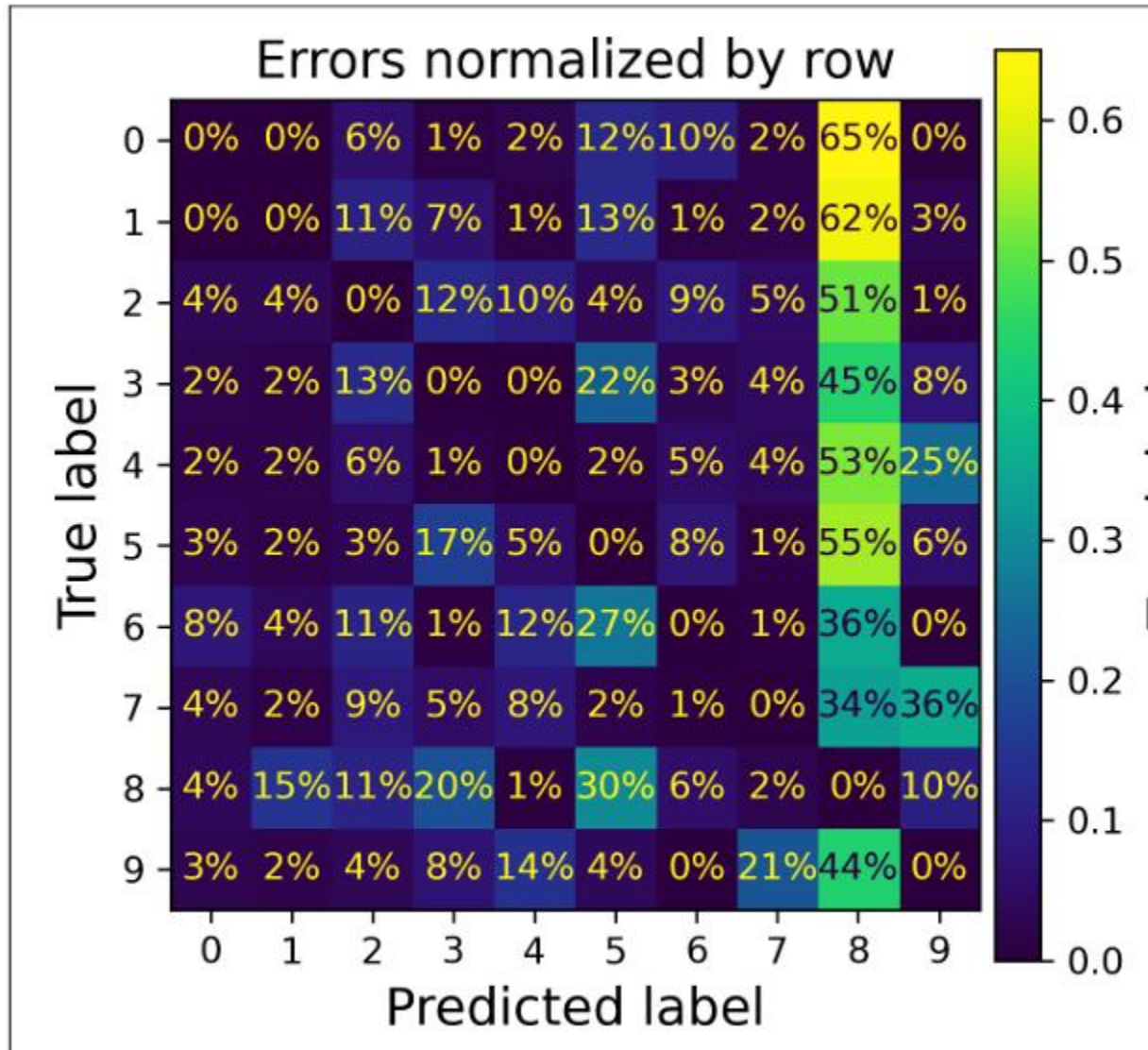
Normalize the confusion matrix by dividing the each by value by the number of true values (divide by the row's sum) (set the normalize parameter = true)

But here the number of errors is not actually clear --> give the correct predictions weight of 0.

```
sample_weight = (y_train_pred != y_train)
```

```
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred,  
sample_weight=sample_weight, normalize="true", values_format=".0%")
```

This will only display the errors

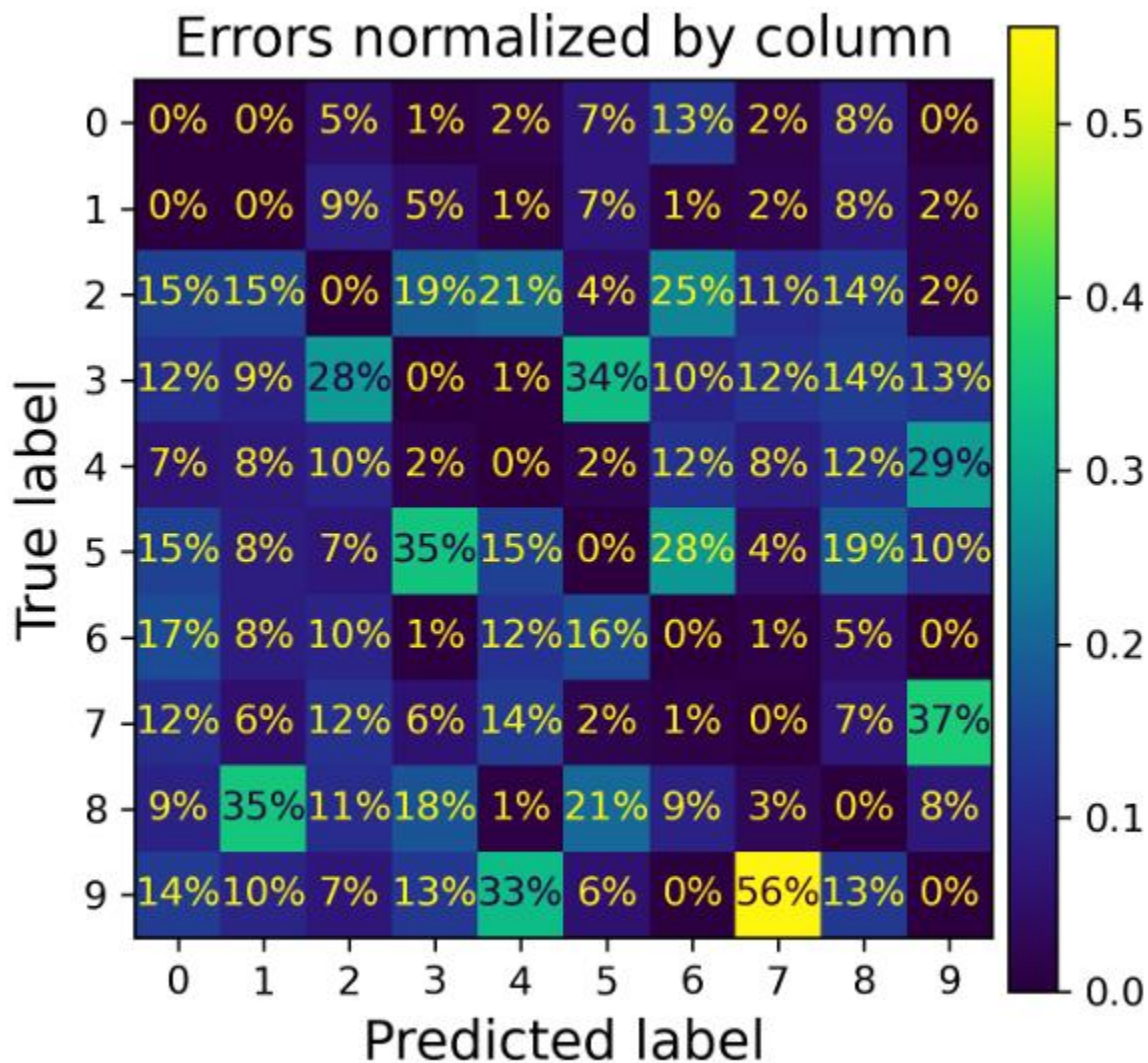


As we can see here, most errors came from incorrectly predicting any number as 8.

These percentages show that % of that number is misclassified as that number

Example row 7 column 9, it means that 36% of the errors on 7 came from misclassifying it as 9

Another way to read a confusion matrix of the errors is by normalizing the column (setting the normalize = “pred”)



This shows 56% of the predicted as 7, were 9

So what are going to do after getting these insights?

Incorrectly predicting any number as 8 , which means that it doesn't know how actually the number 8 looks like , so we might need to add more data about number 8 so that the classifier can distinguish them from the rest

Or write a function to check the number of loops, as only number 8 has 2 loops

Multilabel Classification

You want to get a label for 2 things, not 1

Example for a face recognition classifier i want to see if Alice, bob, and Charlie are all in the picture, so i want the output to be something like True, False, True if given a picture with alice and Charlie

Example 2: If I want to check if a number is large or not based on some logic and if it is odd or not so if given the number 5 the output would be False, True

```
y_train_large = (y_train >= '7') first rule
```

```
y_train_odd = (y_train.astype('int8') % 2 == 1) second rule
```

```
y_multilabel = np.c_[y_train_large, y_train_odd] concatenate them
```

```
knn_clf = KNeighborsClassifier()
```

```
knn_clf.fit(X_train, y_multilabel) train y based on the concatenation
```

To evaluate multilabel:

1-F1 Score:

Get f1-score for each individual label and then get its sum, but this would result in getting equal weights for each label

You choose based on your problem which label to be given a higher weight

But one of the ways is to give a large weight for its support (number of instances with that target label)

If you want to use a model that doesn't natively do multilabel classification, you could use one model for each label you want

But this wouldn't catch the dependencies between them:

a large digit (7, 8, or 9) is twice more likely to be odd than even

Because the odd model doesn't know the output of the large model

To solve this problem --> Chain the models

When a model does a prediction, it uses the input features plus the predictions of the previous models that come before it in the chain

```
chain_clf = ClassifierChain(SVC(), cv=3, random_state=42)
chain_clf.fit(X_train[:2000], y_multilabel[:2000])
```

Multioutput Classification

Generalization of multilabel classification, where each label can be multiclass (each label can have more than 2 possible values)

To illustrate this, let's build a system that removes noise from images. It will take as input a noisy digit image, and it will (hopefully) output a clean digit image, represented as an array of pixel intensities, just like the MNIST images. Notice that the classifier output is multilabel (one label per pixel), and each label can have multiple values (pixel intensity ranges from 0 to 255). This is thus an example of a multioutput classification system.

In this example it's a multilabel because each pixel is a label

Image denoising is a multioutput classification problem because the model predicts multiple labels per input—one for each pixel—and each label can take multiple discrete values corresponding to pixel intensities.